

Assignment 1 — Search Strategies

BFS · DFS · Greedy Best-First · A*

Dr Naila Marir

Instructor: Dr Naila Marir

Total marks: 100

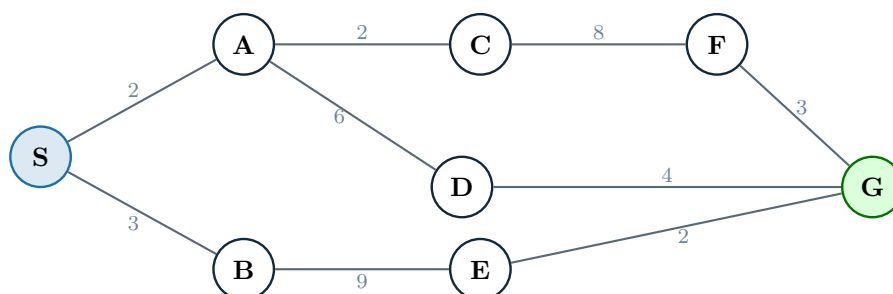
Sections: 3081-1 (CRN 10480), 3081-2 (CRN 10481)

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. S, A, C, G .

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is S and the goal state is G . Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search Queue First in First out

[8 marks]

List the order in which states are expanded by BFS, and give the path it returns.

Expansion:- S, A, B, C, D, E, F, G

Answer:

Path it returns:- $S \rightarrow A \rightarrow D \rightarrow G$

(b) Depth-first search Stack LIFO

[8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

order of expansion :- S, A, C, F, G

Answer:

Path it returns:- S → A → C → F → G

(c) Comparison

[6 marks]

BFS and DFS returned different paths. State which path is shorter *in number of steps*, and explain why BFS is guaranteed to find that path while DFS is not.

BFS is guaranteed to find a path with the fewest number of steps because it explores nodes step by step. DFS is not guaranteed to find the shortest path because it follows one branch deeply before exploring alternatives.

(d) Frontier data structures

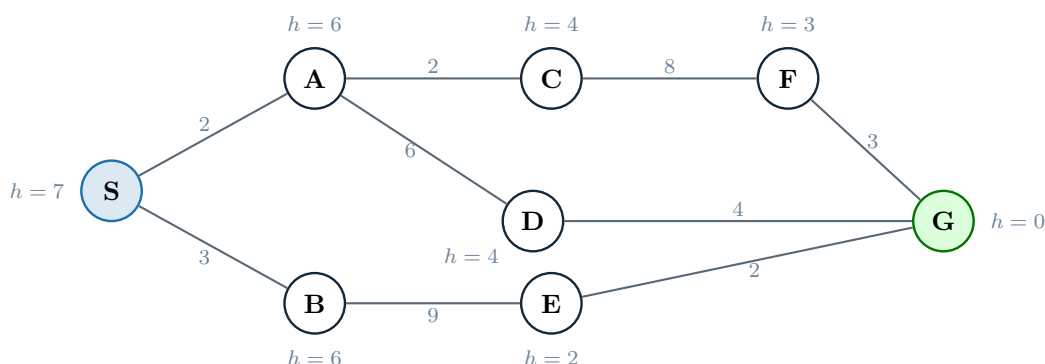
[3 marks]

BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

BFS uses a FIFO queue, while DFS uses a LIFO stack. The queue makes BFS explore level-by-level & the stack makes DFS follow one branch as deeply as possible.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .



(a) Greedy best-first search

[8 marks]

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

order of expansion :- S, A, C, F, G

Answer:

Path returned :- S → A → C → F → G

(b) A* search

[12 marks]

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

Order	Node	$g(n)$	$h(n)$	$f(n) = g(n) + h(n)$
1	S	0	7	7
2	A	2	6	8
3	C	4	4	8
4	B	3	6	9
5	D	8	4	12
6	G	12	0	12

Order of expansion: S, A, C, B, D, G

Path returned: S → A → D → G Total cost: 12

(c) Why they differ

[6 marks]

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Greedy uses only $h(n)$ and ignores the path cost already incurred.
 Answer:
 A* uses $g(n) + h(n)$, so it considers both the cost so far and estimated remaining cost.

3 Admissibility and optimality**(a) Definition**

[5 marks]

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

An admissible heuristic is a heuristic that never overestimates the true minimum cost from a node to the goal. In other words, for every node n , $h(n) \leq h^*(n)$ where $h(n)$ is the estimated cost to the goal and $h^*(n)$ is the actual cheapest cost to the goal.
 Answer:

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G, and decide whether the given h is admissible.

Node	S	A	B	C	D	E	F	G
$h(n)$ given	7	6	6	4	4	2	3	0
$h^*(n)$ true	12	10	11	11	4	2	3	0
$h \leq h^*$	yes	yes	yes	yes	yes	yes	yes	yes

Is h admissible? Yes

If not, name every node that violates the condition: none

(c) Consequence

[6 marks]

Suppose a heuristic overestimates the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why BFS could never suffer from this problem.

IF a heuristic overestimates, A^*
Answer: may return a non-optimal path.
BFS is unaffected because it does not use heuristics

(d) A trivial heuristic

[3 marks]

What does A^* reduce to when $h(n) = 0$ for every node? Justify your answer in one sentence.

when $h(n) = 0$ for every node, A^*
Answer: reduces to Uniform-cost Search because
 $f(n) = g(n) + 0 = g(n)$, so nodes are expanded according to their path cost.

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Optimal?
Breadth-first	FIFO Queue	Yes, when step costs are equal
Depth-first	LIFO stack	No
Greedy best-first	Priority queue by $h(n)$	No
A*	Priority queue by $f(n) = g(n) + h(n)$	Yes, under the required heuristic conditions

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- i. A robot must find *any* route out of a building during a fire. Memory is very limited; the route need not be shortest.

DFS because it uses less memory and can find any route without needing the shortest one.

- ii. A route planner must return the *cheapest* driving route, and a good straight-line-distance estimate is available.

A* because it finds the cheapest route by using both the path cost and a good heuristic.

- iii. A puzzle has uniform step costs, no useful heuristic, and the solution is known to be shallow.

BFS because it is best for shallow solutions with uniform step costs and finds the shortest path.

(c) Short answer

[4 marks]

A* is optimal but is often impractical on large problems. State the reason, and name one property of the search that makes it worse.

A* can be impractical because it uses a lot of memory to store nodes.

A high branching factor makes this worse by generating many more nodes.

Submission. Submit a single PDF to Blackboard by the due date. Handwritten work is acceptable if scanned clearly. Include your name and student ID on the first page.

Q1: 25 Q2: 26 Q3: 24 Q4: 25 **Total: 100 marks**